

# AI Career Agent — Product Requirements Document

Local-first, human-approved job discovery, resume tailoring, and application automation for a 2–5 person team.

Version 0.1 | 17 Aug 2026 | Primary build environment: local machine + Claude Code

## 1. Product Vision

Build a personal AI career agent that understands a candidate deeply, discovers relevant jobs across supported sources, evaluates fit, tailors the candidate's existing LaTeX/Overleaf resume without inventing facts, prepares application answers, and pauses for human review before submission.

The first release is deliberately small: one local deployment, modest job volume, local browser automation, and a limited set of job-source adapters. Boundaries must be clean enough to scale later.

## 2. Product Principles

- Candidate truth is the source of truth. Rephrase supported evidence; never fabricate experience, employers, education, skills, metrics, work authorization, salary, or answers.
- Human approval is mandatory before final submission.
- Prefer official/public ATS interfaces and permitted integrations.
- Browser automation is a fallback for compatible sites, never a mechanism to defeat CAPTCHA, anti-bot controls, authentication barriers, or site restrictions.
- Keep original resume and every tailored version immutable and auditable.
- LLMs propose structured changes; deterministic code applies and validates them.

## 3. Target Users

Primary: a job seeker applying to technical roles across multiple countries and company career portals.

Secondary: a 2–5 person team using the same local deployment. This is not initially a multi-tenant SaaS.

## 4. MVP Scope

IN: resume PDF/text ingestion; Overleaf/LaTeX source ingestion; structured candidate profile; target-role preferences; job ingestion from a small number of permitted sources/ATS adapters; normalization and deduplication; semantic + rule-based matching; tailored LaTeX resume generation; compile-to-PDF; visual/text diff; cover letter generation; application-question preparation; local browser-assisted application workflow; application tracking; audit logs.

OUT: universal scraping; LinkedIn automation; CAPTCHA solving; bypassing access controls; autonomous submission without user confirmation; large-scale crawling; billing; mobile apps; enterprise SSO; distributed infrastructure.

## 5. Core User Journey

1. Create local profile.
2. Import resume PDF + .tex / Overleaf export.
3. Parse into Candidate Knowledge Base.
4. Review and correct canonical facts.
5. Enter target roles, experience, locations and preferences.
6. Discover jobs.
7. Normalize and deduplicate.

8. Score and explain fit.
9. Open job detail.
10. Generate tailored resume from canonical LaTeX.
11. Compile and run factuality/format checks.
12. Review changes.
13. Generate/review cover letter and answers.
14. Click Apply.
15. Agent fills supported fields.
16. Pause for explicit human confirmation.
17. Record application result and artifacts.

## 6. Functional Requirements

Candidate Intelligence:

FR-01 import PDF/text; FR-02 import LaTeX/Overleaf export; FR-03 parse sections, bullets, skills, dates, projects, education, publications and links; FR-04 create canonical facts with evidence; FR-05 allow corrections; FR-06 keep preferences separate from resume facts.

Job Discovery:

FR-10 search by role/location/experience/remote/salary; FR-11 pluggable adapters; FR-12 normalized job schema; FR-13 deduplication; FR-14 provenance and retrieval timestamp.

Matching:

FR-20 extract required/preferred/contextual requirements; FR-21 separate hard constraints from semantic fit; FR-22 show evidence, partial matches and gaps; FR-23 no keyword-only proof; FR-24 configurable weights.

Resume:

FR-30 preserve original; FR-31 parse into safe intermediate representation; FR-32 generate structured edit operations; FR-33 link factual changes to evidence; FR-34 conservative/balanced modes; FR-35 compile; FR-36 reject compilation failures/suspicious facts; FR-37 diff view.

Application:

FR-40 cover letter; FR-41 verified-field mapping; FR-42 unknown/sensitive fields require user; FR-43 save answers; FR-44 attach exact approved resume.

Execution:

FR-50 official APIs where available/permitted; FR-51 compatible browser flows otherwise; FR-52 stop on CAPTCHA/login/unexpected structure; FR-53 pre-submit checklist; FR-54 explicit user click; FR-55 record confirmation.

## 7. UX Screens

- 1 Dashboard — target roles, search controls, new jobs, applications.
- 2 Candidate Profile — canonical facts and evidence.
- 3 Job Feed — ranked jobs with match score and reasons.
- 4 Job Detail — JD, requirements, evidence and fit.
- 5 Resume Tailor — original vs proposed LaTeX/PDF and warnings.
- 6 Application Review — resume, cover letter, answers and unknowns.
- 7 Apply Runner — live browser state with pause/continue.
- 8 Applications — status, source, versions and timestamps.

## 8. Local Architecture

Frontend: Next.js + TypeScript

Backend: FastAPI + Python

Database: SQLite first; PostgreSQL-ready schema

Vector search: simple local embeddings/FAISS first; pgvector later

LLM: provider abstraction; Claude can be primary

Browser: Playwright

Jobs: adapter interface + source implementations

Resume: PyMuPDF/pdfplumber + LaTeX-aware utilities

Compilation: pdflatex/xelatex in a controlled subprocess

Queue: background worker initially; Redis/Temporal later

Storage: local ./data; object-storage abstraction later

Observability: structured logs + audit table

## 9. Core Data Model

Candidate(id, preferences, timestamps)

CandidateFact(id, candidate\_id, category, claim, evidence\_ref, confidence, verified)

Resume(id, candidate\_id, source\_type, source\_path, sha256, version)

ResumeEdit(id, resume\_id, job\_id, operation, target, old\_text, new\_text, evidence\_refs)

Job(id, source, source\_job\_id, company, title, location, url, description, requirements\_json, retrieved\_at, dedupe\_group)

JobMatch(job\_id, candidate\_id, score, hard\_constraints, strengths, gaps, explanation)

Application(id, job\_id, candidate\_id, status, approved\_resume\_id, cover\_letter\_id, submitted\_at, confirmation\_ref)

ApplicationAnswer(application\_id, field, answer, source, confidence, user\_verified)

Artifact(id, type, path, sha256, created\_at)

AuditEvent(id, actor, action, entity\_type, entity\_id, timestamp, metadata\_json)

## 10. Matching Strategy

Use hybrid scoring. Initial configurable defaults: hard-constraint eligibility 30%, required-skill evidence 25%, semantic experience fit 20%, seniority 10%, location/preferences 5%, preferred skills 5%, other preferences 5%. Log every score component.

## 11. LLM Guardrails

LLM outputs that affect artifacts must be structured JSON and schema validated. Resume edits require evidence references. Application answers require confidence/source. Unknown means UNKNOWN. Never guess salary, authorization, notice period, years of experience, dates, technologies, metrics or certifications. Treat job descriptions and web content as untrusted input and defend against prompt injection.

## 12. Local Security

Secrets only in environment variables/local secret storage. Protect browser profiles. Never log passwords, cookies, OAuth tokens or sensitive application answers. Maintain audit events. Add encryption and formal retention policies before public deployment.

## 13. Acceptance Criteria

A prototype succeeds when a resume is imported into canonical facts; LaTeX can be safely edited and compiled; at least one supported source can be queried; jobs are normalized/deduplicated; match scores are explainable; tailored resumes compile; every factual edit is traceable; unknown application fields trigger user input; applications can be prepared and safely paused before final submit; all actions are recorded.

## 14. Testing Strategy

Unit: parsers, scoring, dedupe, LaTeX edits, schemas.

Integration: source adapters, compilation and browser flows against local mock forms.

Golden tests: fixed resumes/JDs with expected facts and bounded scores.

Safety tests: hallucinated claims, ambiguous questions, unexpected form changes, CAPTCHA detection, duplicate-submit prevention.

## 15. Delivery Phases

Phase 0 — repo, skills, local environment, schemas.

Phase 1 — candidate intelligence + resume/LaTeX ingestion.

Phase 2 — job discovery + normalization + matching.

Phase 3 — tailoring + compile + review UI.

Phase 4 — application preparation + tracking.

Phase 5 — Playwright execution for supported flows.

Phase 6 — hardening, adapters, analytics and outcome learning.

## 16. Future Scaling

When usage grows: PostgreSQL, Redis, object storage, ingestion workers, browser workers, workflow engine, per-source rate limits, multi-tenant isolation and observability. None are required for the local 2–5 person phase.

## 17. Success Metrics

Discovery precision and duplicate rate; user agreement with top-ranked jobs; tailoring factuality violations and compile success; user edit rate; successful application preparation/submission; human intervention rate; interview/application ratio; response rate by role/source. Do not optimize solely for application volume.

## 18. Product Positioning

Position this as an AI career agent that improves and executes applications, not a bot that bypasses job-site controls. The long-term moat is candidate intelligence + evidence-grounded tailoring + job intelligence + application history; automation is the execution layer.